

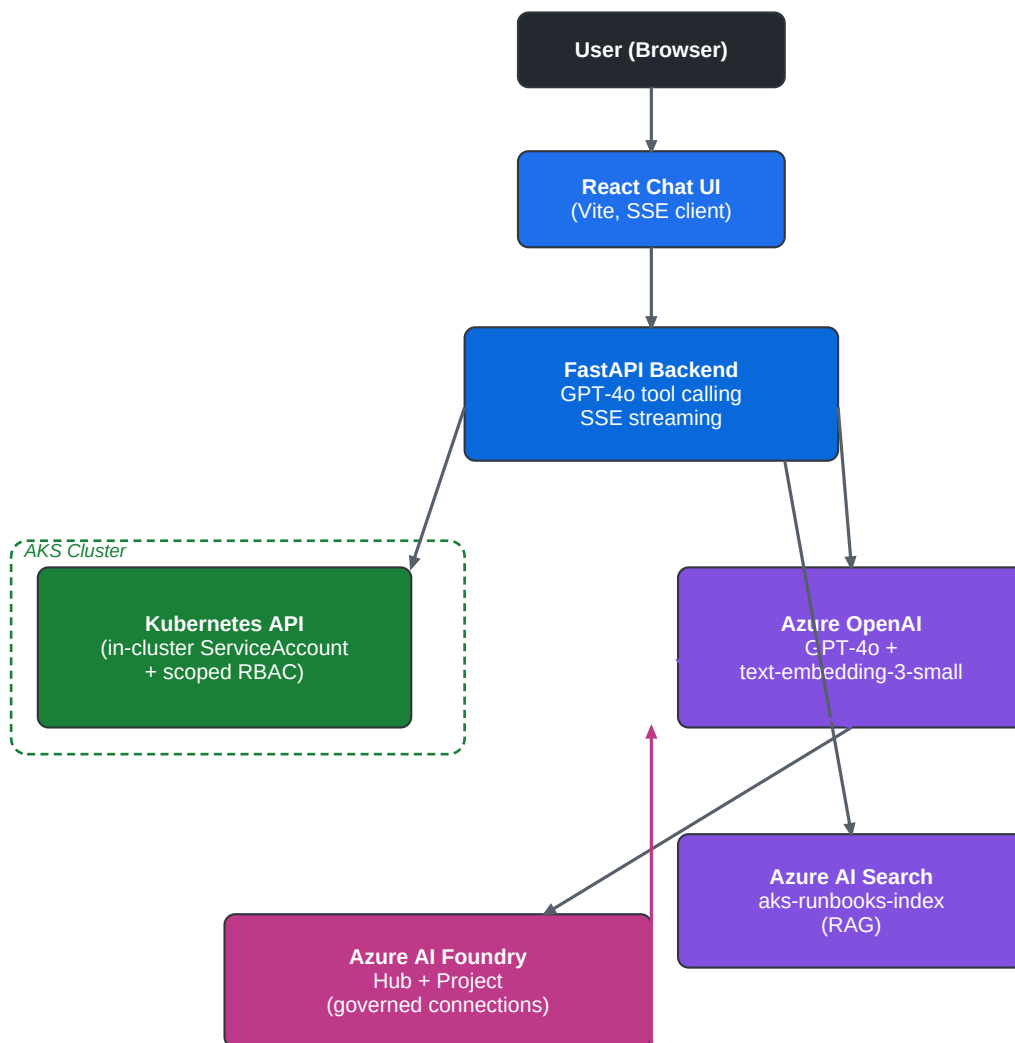
# AKS Copilot

AI Chatbot for AKS Troubleshooting & Kubernetes Object Management

An Azure OpenAI (GPT-4o) powered chatbot with live, RBAC-scoped access to an AKS cluster. It diagnoses real-time failures (CrashLoopBackOff, pending pods, networking issues, node pressure), explains root cause, and can create or modify Kubernetes objects (Pods, Deployments, Services, Secrets, ConfigMaps) on request. Troubleshooting guidance is grounded via Retrieval-Augmented Generation (RAG) over internal runbooks using Azure AI Search, with Azure AI Foundry providing the governed Hub/Project connections.

Repository: [github.com/ramamishra7262/aks\\_chat\\_bot](https://github.com/ramamishra7262/aks_chat_bot)

# 1. Architecture Overview



The React frontend streams chat over Server-Sent Events (SSE) to the FastAPI backend. The backend calls Azure OpenAI GPT-4o with a set of function-calling tools that wrap the Kubernetes Python client. Before each turn, it queries Azure AI Search for relevant runbook excerpts (RAG) and injects them as additional context. Azure AI Foundry Hub/Project hosts governed AAD connections to both Azure OpenAI and Azure AI Search.

## Component summary

| Component      | Technology                     | Purpose                                                          |
|----------------|--------------------------------|------------------------------------------------------------------|
| Chat UI        | React + Vite                   | Streaming chat, tool-call viewer, namespace selector, dark theme |
| Backend API    | FastAPI (Python)               | Azure OpenAI tool-calling orchestration, SSE streaming, RAG      |
| LLM            | Azure OpenAI GPT-4o            | Reasoning, root-cause analysis, tool/function calling            |
| Embeddings     | text-embedding-3-small         | Vectorizes runbooks + queries for RAG                            |
| RAG store      | Azure AI Search                | Hybrid vector + keyword search over aks-runbooks-index           |
| Governance     | Azure AI Foundry Hub + Project | Shared AAD connections, project tracking                         |
| Cluster access | Kubernetes Python client       | In-cluster ServiceAccount, scoped RBAC, namespace allowlist      |

|                      |                                 |                                                 |
|----------------------|---------------------------------|-------------------------------------------------|
| Container registry   | Azure Container Registry        | Stores backend/frontend images                  |
| Hosting (in-cluster) | AKS Deployments + HPA + Ingress | Primary hosting path inside the target cluster  |
| Hosting (alt.)       | Azure Container Apps            | Optional path for managing a remote AKS cluster |
| Secrets/Identity     | Key Vault, Workload Identity    | Secret storage, federated AAD identity for pods |
| Observability        | Log Analytics + App Insights    | Logs, metrics, tracing for the chatbot          |

## Tool-calling capabilities

### Diagnostic (read-only) tools:

- `get_pods` - list pods, status, restarts, node placement
- `get_pod_logs` - fetch current/previous container logs
- `describe_pod` - conditions and container states (kubectl describe equivalent)
- `get_events` - recent Warning/Normal events for root-cause analysis
- `get_deployments` / `get_services` / `get_nodes` - inventory and rollout status
- `check_pod_health` - aggregate healthy vs unhealthy pod summary per namespace

### Mutation tools (object creation & remediation):

- `create_pod` / `create_deployment` / `create_service` / `create_secret` / `create_configmap`
- `scale_deployment` - change replica count
- `restart_deployment` - rolling restart via annotation patch
- `delete_resource` - delete Pod/Deployment/Service/Secret/ConfigMap
- All `create_*` tools support `dry_run` for server-side validation only

## Safety controls

- `KUBE_NAMESPACE_ALLOWLIST` restricts which namespaces tools can read/write
- `ENABLE_MUTATIONS=false` disables all create/delete/scale/restart tools (diagnostics still work)
- RBAC ClusterRole (k8s/rbac.yaml) grants only the verbs the tools actually use - no cluster-admin
- Dedicated ServiceAccount with Azure Workload Identity for AAD-based access to OpenAI/AI Search

## 2. Repository Layout

```
backend/ FastAPI app (Azure OpenAI tool-calling, K8s client, RAG, SSE)
app/core/config.py Settings (env vars)
app/models/schemas.py Pydantic request/response models
app/services/kubernetes_service.py K8s client wrapper (in-cluster or kubeconfig)
app/services/openai_service.py Azure OpenAI streaming + tool-calling
app/services/search_service.py Azure AI Search RAG queries
app/services/streaming_service.py SSE formatting helpers
app/tools/k8s_diagnostic_tools.py Read-only tools
app/tools/k8s_mutation_tools.py Mutating tools
app/api/chat.py POST /api/chat/stream (SSE)
app/api/health.py /healthz, /readyz
tests/ pytest unit tests
Dockerfile

frontend/ React + Vite chat UI (dark theme, streaming, tool-call viewer)
src/components/ ChatWindow, MessageBubble, ToolCallDisplay, KubernetesObjectPreview
src/services/api.js SSE client
Dockerfile, nginx.conf

infra/
bicep/main.bicep Orchestrates everything below
bicep/ai-hub.bicep Azure AI Foundry Hub (+ OpenAI/Search connections)
bicep/ai-project.bicep Azure AI Foundry Project
bicep/ai-search.bicep Azure AI Search service
bicep/container-apps.bicep Optional Container Apps hosting path
bicep/modules/openai.bicep GPT-4o + text-embedding-3-small deployments
bicep/modules/keyvault.bicep, storage.bicep, container-registry.bicep, log-analytics.bicep
bicep/modules/role-assignments.bicep Least-privilege RBAC for OpenAI + AI Search
runbooks/ Markdown troubleshooting runbooks indexed for RAG
scripts/setup.sh One-shot infra bootstrap + GitHub OIDC service principal
scripts/index-k8s-docs.py Embeds + uploads runbooks to Azure AI Search

k8s/ In-cluster deployment manifests
namespace.yaml, serviceaccount.yaml, rbac.yaml, configmap.yaml, secret.yaml
deployment.yaml, service.yaml, hpa.yaml, ingress.yaml

.github/workflows/
build-push.yml Test, build, scan (Trivy), push images to GHCR
infra-deploy.yml Deploy Azure AI infra via Bicep (OIDC)
index-runbooks.yml (Re)build the AI Search RAG index
app-deploy.yml Deploy to AKS (kubectl apply + rollout + smoke test)
```

## 3. Complete Deployment Steps

### Step 1 - Provision Azure AI infrastructure

Run the bootstrap script, which deploys infra/bicep/main.bicep and creates a GitHub Actions OIDC service principal with federated credentials.

```
cd infra/scripts
./setup.sh aks-copilot-rg eastus akscopilot
```

This provisions:

- Azure OpenAI account with gpt-4o and text-embedding-3-small deployments
- Azure AI Search service
- Azure AI Foundry Hub + Project (with AAD connections to OpenAI and AI Search)
- Azure Container Registry, Key Vault, Storage Account, Log Analytics + Application Insights
- (Optional) Container Apps environment + backend/frontend apps
- Role assignments: Cognitive Services OpenAI User, Search Index Data Reader, Search Service Contributor on the Hub/Project/Container App identities

### Step 2 - Index AKS troubleshooting runbooks (RAG)

```
export AZURE_OPENAI_ENDPOINT=... AZURE_OPENAI_API_KEY=...
export AZURE_SEARCH_ENDPOINT=... AZURE_SEARCH_API_KEY=...
python infra/scripts/index-k8s-docs.py
```

This creates the vector-enabled aks-runbooks-index, chunks each markdown runbook in infra/runbooks/, embeds each chunk, and uploads them to Azure AI Search. It also runs automatically via the index-runbooks.yml workflow whenever infra/runbooks/ changes.

### Step 3 - Configure GitHub repository secrets

| Secret                                                    | Description                                |
|-----------------------------------------------------------|--------------------------------------------|
| AZURE_CLIENT_ID / AZURE_TENANT_ID / AZURE_SUBSCRIPTION_ID | OpenAI federated credentials from setup.sh |
| AZURE_RESOURCE_GROUP                                      | Resource group created by setup.sh         |
| AKS_CLUSTER_NAME                                          | Target AKS cluster name                    |
| AZURE_OPENAI_ENDPOINT / AZURE_OPENAI_API_KEY              | From the openAi Bicep module outputs       |
| AZURE_SEARCH_ENDPOINT / AZURE_SEARCH_API_KEY              | From the aiSearch Bicep module outputs     |

### Step 4 - Build, scan and push images (CI)

On push to main, build-push.yml runs backend pytest tests, builds the frontend, then builds, Trivy-scans, and pushes aks-copilot-backend and aks-copilot-frontend images to GHCR (ghcr.io/ramamishra7262/...).

### Step 5 - Deploy the chatbot to AKS

app-deploy.yml authenticates to Azure via OIDC, fetches AKS credentials, and applies the manifests in k8s/ in order:

- namespace.yaml - creates the aks-copilot namespace
- serviceaccount.yaml - ServiceAccount with Workload Identity annotation

- rbac.yaml - ClusterRole/ClusterRoleBinding scoped to tool verbs only
- configmap.yaml - non-secret configuration (deployment names, allowlists)
- secret.yaml (via kubectl create secret, populated from GitHub secrets) - Azure OpenAI / AI Search keys
- deployment.yaml - backend (FastAPI) + frontend (React/nginx) Deployments
- service.yaml - ClusterIP services for backend and frontend
- hpa.yaml - HorizontalPodAutoscalers (CPU-based, 70% target)
- ingress.yaml - NGINX ingress with TLS via cert-manager ClusterIssuer

The pipeline then waits for both rollouts to complete and runs a /healthz smoke test pod before finishing.

## Step 6 - Local development (optional)

```
# Backend
cd backend
cp .env.example .env # fill in Azure OpenAI / AI Search keys
pip install -r requirements.txt
uvicorn app.main:app --reload

# Frontend
cd frontend
cp .env.example .env
npm install
npm run dev

# Or run both:
docker compose up --build
```

## Step 7 - Verify

- Backend tests: `cd backend && pytest -q` (10 tests covering health, K8s tool dispatch, RBAC guardrails)
- GET /healthz reports `kubernetes_connected`, `azure_openai_configured`, `azure_search_configured`
- Open the frontend, ask 'Show me unhealthy pods in default' and confirm tool-call cards render with results
- Try 'Create a ConfigMap named feature-flags with KEY=true' to confirm mutation tools and `dry_run` work

Repository: [github.com/ramamishra7262/aks\\_chat\\_bot](https://github.com/ramamishra7262/aks_chat_bot)